



ULGEN Trello Clone - Kurulum ve Çalıştırma Kılavuzu

Bu doküman, ULGEN Trello Clone projesini sıfırdan kurmak ve çalıştırmak için gereken tüm adımları içermektedir.



İçindekiler

1. [Sistem Gereksinimleri](#)
2. [Gerekli Yazılımların Kurulumu](#)
3. [Proje Kurulumu](#)
4. [Veritabanı Konfigürasyonu](#)
5. [Environment Variables Konfigürasyonu](#)
6. [Veritabanı Migrasyonları ve Seed Data](#)
7. [Uygulamayı Çalıştırma](#)
8. [İlk Giriş ve Kullanım](#)
9. [Sık Karşılaşılan Sorunlar ve Çözümleri](#)
10. [Komut Referansı](#)
11. [Ek Bilgiler](#)



Sistem Gereksinimleri

Minimum Donanım Gereksinimleri

- **İşlemci:** 2 GHz dual-core veya üzeri
- **RAM:** 4 GB (8 GB önerilir)
- **Disk Alanı:** 2 GB boş alan
- **İnternet Bağlantısı:** Paket indirmeleri için gerekli

Desteklenen İşletim Sistemleri

- ☒ **Windows 11** (Windows 10 da desteklenir)
- ☒ **macOS** (10.15 Catalina veya üzeri)
- ☒ **Linux** (Ubuntu 20.04+ veya benzeri)



Gerekli Yazılımların Kurulumu

1. Node.js (v18 veya üzeri)

Node.js, projenin çalışması için gerekli JavaScript runtime ortamıdır.



Windows Kurulumu

1. [Node.js resmi web sitesine](https://nodejs.org/) (https://nodejs.org/) gidin

2. **LTS (Long Term Support)** versiyonunu indirin (v18 veya üzeri)
3. İndirilen `.msi` dosyasını çalıştırın
4. Kurulum sihirbazında:
 - “Next” butonuna tıklayın
 - Lisans sözleşmesini kabul edin
 - Varsayılan kurulum yolunu kullanın: `C:\Program Files\nodejs\`
 - “Automatically install necessary tools” seçeneğini işaretleyin
 - “Install” butonuna tıklayın
5. Kurulum tamamlandıktan sonra **bilgisayarınızı yeniden başlatın**

Kurulumu Doğrulama:

```
node --version
npm --version
```

Çıktı şuna benzer olmalı:

```
v18.x.x
9.x.x
```



macOS Kurulumu

Yöntem 1: Resmi İndirme (Önerilen)

1. [Node.js resmi web sitesine](https://nodejs.org/) (<https://nodejs.org/>) gidin
2. **LTS (Long Term Support)** versiyonunu indirin (v18 veya üzeri)
3. İndirilen `.pkg` dosyasını çalıştırın
4. Kurulum sihirbazını takip edin
5. Şifrenizi girmeniz istenebilir

Yöntem 2: Homebrew ile Kurulum

```
# Homebrew kurulu değilse önce kurun
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Node.js'i kurun
brew install node@18
```

Kurulumu Doğrulama:

```
node --version
npm --version
```

2. pnpm (Paket Yöneticisi)

pnpm, npm'e göre daha hızlı ve disk alanı tasarruflu bir paket yöneticisidir.



Windows Kurulumu

PowerShell'i Yönetici Olarak Açın ve şu komutu çalıştırın:

```
npm install -g pnpm
```

macOS/Linux Kurulumu

Terminal'i açın ve şu komutu çalıştırın:

```
npm install -g pnpm
```

Kurulumu Doğrulama:

```
pnpm --version
```

Çıktı şuna benzer olmalı: `8.x.x` veya üzeri

3. PostgreSQL (Veritabanı)

PostgreSQL, projenin veritabanı sistemidir.

Windows Kurulumu

1. [PostgreSQL resmi web sitesine](https://www.postgresql.org/download/windows/) (<https://www.postgresql.org/download/windows/>) gidin
2. **"Download the installer"** linkine tıklayın
3. En son sürümü (15.x veya 16.x) indirin
4. İndirilen `.exe` dosyasını çalıştırın
5. Kurulum sihirbazında:
 - **Installation Directory:** Varsayılan bırakın (`C:\Program Files\PostgreSQL\16`)
 - **Components:** Tümünü seçili bırakın (PostgreSQL Server, pgAdmin 4, Command Line Tools)
 - **Data Directory:** Varsayılan bırakın
 - **Password:** Güçlü bir şifre belirleyin ve **NOT EDİN** (örn: `postgres123`)
 - **Port:** Varsayılan `5432` portunu kullanın
 - **Locale:** Varsayılan bırakın
6. "Next" ve "Finish" ile kurulumu tamamlayın

Kurulumu Doğrulama:

Windows Arama'da "SQL Shell (psql)" yazın ve açın:

```
Server [localhost]: (Enter tuşuna basın)
Database [postgres]: (Enter tuşuna basın)
Port [5432]: (Enter tuşuna basın)
Username [postgres]: (Enter tuşuna basın)
Password: (Kurulumda belirlediğiniz şifreyi girin)
```

Başarılı giriş yaptıysanız `postgres=#` promptunu göreceksiniz.

macOS Kurulumu

Yöntem 1: Postgres.app (En Kolay - Önerilen)

1. [Postgres.app](https://postgresapp.com/) (<https://postgresapp.com/>) sitesine gidin
2. "Download" butonuna tıklayın

3. İndirilen `.dmg` dosyasını açın
4. Postgres.app'i Applications klasörüne sürükleyin
5. Applications'dan Postgres.app'i açın
6. "Initialize" butonuna tıklayın
7. Fil simgesi menü çubuğunda görünecek

PATH'e Ekleme (Terminal komutları için):

```
echo 'export PATH="/Applications/Postgres.app/Contents/Versions/latest/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

Yöntem 2: Homebrew ile Kurulum

```
# PostgreSQL'i kurun
brew install postgresql@15

# PostgreSQL servisini başlatın
brew services start postgresql@15

# PATH'e ekleyin
echo 'export PATH="/opt/homebrew/opt/postgresql@15/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

Kurulumu Doğrulama:

```
psql --version
```

Çıktı şuna benzer olmalı: `psql (PostgreSQL) 15.x`

4. Git (Opsiyonel)

Proje dosyalarını GitHub'dan klonlamak için gereklidir.



Windows Kurulumu

1. [Git for Windows](https://git-scm.com/download/win) (<https://git-scm.com/download/win>) sitesine gidin
2. İndirilen `.exe` dosyasını çalıştırın
3. Varsayılan ayarlarla kurulumu tamamlayın



macOS Kurulumu

```
# Homebrew ile
brew install git

# Veya Xcode Command Line Tools ile
xcode-select --install
```

Kurulumu Doğrulama:

```
git --version
```



Proje Kurulumu

Adım 1: Proje Dosyalarını Edinme

Seenek A: Git ile Klonlama (Önerilen)

```
# Terminal veya Command Prompt'u açın
cd ~
git clone <repository-url> trello-clone
cd trello-clone
```

Seenek B: ZIP Dosyasından

1. Proje ZIP dosyasını indirin
2. İstedığınız bir konuma çıkartın
3. Terminal/Command Prompt'ta proje klasörüne gidin:

```
# Windows
cd C:\Users\YourUsername\trello-clone

# macOS/Linux
cd ~/trello-clone
```

Adım 2: Dependencies Kurulumu

Proje klasöründeyken şu komutu çalıştırın:

```
pnpm install
```

Bu komut:

- package.json dosyasındaki tüm bağımlılıkları indirir
- node_modules klasörünü oluşturur
- Yaklaşık 2-5 dakika sürebilir (internet hızınıza bağlı)

Beklenen Çıktı:

```
Packages: +XXX
+++++
Progress: resolved XXX, reused XXX, downloaded XX, added XXX, done
```



Veritabanı Konfigürasyonu

Adım 1: PostgreSQL Servisini Başlatma



Windows

PostgreSQL kurulumdan sonra otomatik başlar. Kontrol etmek için:

1. Windows Arama'da "Services" yazın ve açın
2. "postgresql-x64-16" servisini bulun

3. Status: "Running" olmalı
4. Değilse, sağ tıklayıp "Start" seçin

Veya Command Prompt'ta:

```
net start postgresql-x64-16
```

 macOS**Postgres.app kullanıyorsanız:**

- Uygulamayı açın, otomatik başlar

Homebrew kullanıyorsanız:

```
brew services start postgresql@15
```

Servisi Kontrol Etme:

```
brew services list
```

Adım 2: Veritabanı Oluşturma**Yöntem 1: Terminal/Command Line ile (Önerilen)**

```
# PostgreSQL kullanıcısı olarak bağlanın
psql -U postgres

# Veritabanını oluşturun
CREATE DATABASE ulgen_db;

# Veritabanını listeleyin (kontrol için)
\l

# Çıkış yapın
\q
```

macOS (Postgres.app) için:

```
# Varsayılan kullanıcı adınızla bağlanın
psql

# Veritabanını oluşturun
CREATE DATABASE ulgen_db;

# Çıkış
\q
```

Yöntem 2: pgAdmin ile (GUI)

1. pgAdmin 4'ü açın (Windows'ta başlat menüsünden)
2. Sol panelde "Servers" > "PostgreSQL 16" > sağ tık > "Connect"
3. Şifrenizi girin
4. "Databases" üzerine sağ tık > "Create" > "Database"
5. Database name: `ulgen_db`

6. “Save” butonuna tıklayın

Adım 3: Connection String Hazırlama

Connection string formatı:

```
postgresql://[kullanıcı]:[şifre]@[host]:[port]/[veritabanı_adı]
```

Örnekler:

```
# Windows (varsayılan kurulum)
postgresql://postgres:postgres123@localhost:5432/ulgen_db

# macOS (Postgres.app - şifresiz)
postgresql://yourusername@localhost:5432/ulgen_db

# macOS (Homebrew)
postgresql://yourusername@localhost:5432/ulgen_db
```

Not: yourusername yerine macOS kullanıcı adınızı yazın. Bulmak için:

```
whoami
```



Environment Variables Konfigürasyonu

Adım 1: .env Dosyası Oluşturma

Proje klasöründe .env.example dosyasını .env olarak kopyalayın:



Windows (PowerShell)

```
Copy-Item .env.example .env
```



Windows (Command Prompt)

```
copy .env.example .env
```



macOS/Linux

```
cp .env.example .env
```

Adım 2: .env Dosyasını Düzenleme

.env dosyasını bir metin editörü ile açın (VS Code, Notepad++, vb.)

```
# VS Code ile açmak için
code .env

# Veya herhangi bir editör ile
notepad .env # Windows
open -e .env # macOS
```

Adım 3: Environment Variables'ları Ayarlama

1. DATABASE_URL

Veritabanı bağlantı string'inizi girin:

```
DATABASE_URL="postgresql://postgres:postgres123@localhost:5432/ulgen_db"
```

Kendi bilgilerinize göre düzenleyin:

- postgres → PostgreSQL kullanıcı adınız
- postgres123 → PostgreSQL şifreniz
- localhost → Veritabanı sunucu adresi (genelde localhost)
- 5432 → PostgreSQL portu (varsayılan)
- ulgen_db → Oluşturduğunuz veritabanı adı

2. NEXTAUTH_SECRET

Bu, NextAuth.js'in oturum şifrelemesi için kullandığı gizli anahtardır.

Güvenli bir secret oluşturma:

```
# Terminal/Command Prompt'ta
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
```

Çıktıyı kopyalayın ve .env dosyasına yapıştırın:

```
NEXTAUTH_SECRET="oluşturulan-gizli-anahtar-buraya"
```

Örnek:

```
NEXTAUTH_SECRET="Xy7+mK9pQw3nR5tV8uZ2aB4cD6eF1gH3jL5mN7pQ9sT2vW4xY6zA8bC0dE2fG4h="
```

3. NEXTAUTH_URL

Uygulamanızın çalıştığı URL:

```
# Development (geliştirme) için
NEXTAUTH_URL="http://localhost:3000"

# Production (canlı) için (deploy ettikten sonra)
NEXTAUTH_URL="https://yourdomain.com"
```

Şimdilik development için bırakın:

```
NEXTAUTH_URL="http://localhost:3000"
```


4. GOOGLE_CLIENT_ID ve GOOGLE_CLIENT_SECRET (Opsiyonel)

Google ile giriş özelliği için gereklidir. **Şimdilik boş bırakabilirsiniz**, email/password ile giriş yapabilirsiniz.

Google OAuth Credentials Alma (İsteğe Bağlı):

1. [Google Cloud Console](https://console.cloud.google.com/) (https://console.cloud.google.com/) gidin
2. Yeni bir proje oluşturun veya mevcut birini seçin
3. Sol menüden **"APIs & Services"** > **"Credentials"** seçin
4. **"Create Credentials"** > **"OAuth client ID"** tıklayın
5. Application type: **"Web application"** seçin
6. Name: `ULGEN Trello Clone` yazın
7. **Authorized JavaScript origins:**
`http://localhost:3000`
8. **Authorized redirect URIs:**
`http://localhost:3000/api/auth/callback/google`
9. **"Create"** butonuna tıklayın
10. Açılan pencereden **Client ID** ve **Client Secret**'i kopyalayın

`.env` dosyasına ekleyin:

```
GOOGLE_CLIENT_ID="your-client-id.apps.googleusercontent.com"
GOOGLE_CLIENT_SECRET="your-client-secret"
```

Örnek .env Dosyası

```
# Database
DATABASE_URL="postgres://postgres:postgres123@localhost:5432/ulgen_db"

# NextAuth
NEXTAUTH_SECRET="Xy7+mK9pQw3nR5tV8uZ2aB4cD6eF1gH3jL5mN7pQ9sT2vW4xY6zA8bC0dE2fG4h="
NEXTAUTH_URL="http://localhost:3000"

# Google OAuth (Opsiyonel)
GOOGLE_CLIENT_ID=""
GOOGLE_CLIENT_SECRET=""
```

⚠ **Önemli:** `.env` dosyasını asla Git'e commit etmeyin! Bu dosya `.gitignore` 'da zaten listelenmiştir.



Veritabanı Migrasyonları ve Seed Data

Adım 1: Prisma Client Generate

Prisma Client'ı oluşturun (TypeScript type definitions için gerekli):

```
pnpm db:generate
```

Bu komut:

- `prisma/schema.prisma` dosyasını okur

- TypeScript için type-safe database client oluşturur
- `node_modules/@prisma/client` klasörüne yazar

Beklenen Çıktı:

✓ Generated Prisma Client `to` `./node_modules/@prisma/client`

Adım 2: Schema'yı Veritabanına Push Etme

Veritabanı tablolarını oluşturun:

```
pnpm db:push
```

Bu komut:

- `prisma/schema.prisma` 'daki modelleri okur
- PostgreSQL veritabanında tabloları oluşturur
- İlişkileri, indeksleri ve kısıtlamaları ayarlar

Beklenen Çıktı:

🚀 Your database is now in sync with your Prisma schema.

Oluşturulan Tablolar:

- User - Kullanıcı bilgileri
- Account - OAuth hesap bağlantıları
- Session - Oturum yönetimi
- Board - Panolar
- List - Listeler
- Card - Kartlar
- Comment - Yorumlar
- Attachment - Dosya ekleri
- Label - Etiketler
- Activity - Aktivite logları

Adım 3: Demo Data Yükleme (Opsiyonel)

Demo kullanıcı ve örnek board'lar oluşturun:

```
pnpm db:seed
```

Bu komut:

- Demo kullanıcı oluşturur (email: `demo@ulgen.com` , password: `demo123`)
- Örnek board'lar, listeler ve kartlar ekler
- Test için hazır veri sağlar

Beklenen Çıktı:

🌱 Seeding database...
 ✓ Demo user created
 ✓ Sample boards created
 🎉 Database seeded successfully!

⚠ **Not:** Seed komutu sadece bir kez çalıştırılmalıdır. Tekrar çalıştırırsanız hata alabilirsiniz (duplicate user).

Uygulamayı Çalıştırma

Development Mode (Geliştirme Modu)

Geliştirme sırasında kullanılır, hot-reload özelliği vardır:

```
pnpm dev
```

Bu komut:

- Next.js development server'ı başlatır
- `http://localhost:3000` adresinde çalışır
- Dosya değişikliklerini otomatik algılar ve sayfayı yeniler
- Detaylı hata mesajları gösterir

Beklenen Çıktı:

```
▲ Next.js 14.x.x
- Local:      http://localhost:3000
- Network:    http://192.168.x.x:3000

✓ Ready in 3.2s
```

Tarayıcınızda açın: <http://localhost:3000> (`http://localhost:3000`)

Production Mode (Canlı Mod)

Optimize edilmiş production build için:

```
# Önce build edin
pnpm build

# Sonra başlatın
pnpm start
```

Build komutu:

- Kodu optimize eder ve minify eder
- Static sayfaları pre-render eder
- Production için hazır hale getirir

Start komutu:

- Production server'ı başlatır
- `http://localhost:3000` adresinde çalışır

Port Değiştirme

Farklı bir port kullanmak isterseniz:

```
# Windows (PowerShell)
$env:PORT=3001; pnpm dev

# Windows (Command Prompt)
set PORT=3001 && pnpm dev

# macOS/Linux
PORT=3001 pnpm dev
```

İlk Giriş ve Kullanım

Adım 1: Uygulamayı Tarayıcıda Açma

1. Tarayıcınızı açın (Chrome, Firefox, Safari, Edge)
2. Adres çubuğuna yazın: `http://localhost:3000`
3. Ana sayfa yüklenecektir

Adım 2: Demo Hesapla Giriş Yapma

Seed data yüklediyseniz, demo hesapla giriş yapabilirsiniz:

1. **“Sign In”** butonuna tıklayın
2. Giriş bilgilerini girin:
 - **Email:** `demo@ulgen.com`
 - **Password:** `demo123`
3. **“Sign In”** butonuna tıklayın
4. Dashboard’a yönlendirileceksiniz

Adım 3: Yeni Hesap Oluşturma

Kendi hesabınızı oluşturmak için:

1. **“Sign Up”** veya **“Create Account”** linkine tıklayın
2. Formu doldurun:
 - **Name:** Adınız Soyadınız
 - **Email:** `email@example.com`
 - **Password:** Güçlü bir şifre (min. 6 karakter)
3. **“Sign Up”** butonuna tıklayın
4. Otomatik olarak giriş yapılacak ve dashboard’a yönlendirileceksiniz

Adım 4: İlk Board Oluşturma

1. Dashboard’da **“Create New Board”** butonuna tıklayın
2. Board bilgilerini girin:
 - **Title:** Proje Adı (örn: “Web Sitesi Geliştirme”)
 - **Description:** Kısa açıklama (opsiyonel)
 - **Background:** Arka plan rengi veya resmi seçin
3. **“Create”** butonuna tıklayın
4. Yeni board’unuz açılacak

Adım 5: Liste ve Kart Ekleme

Liste Ekleme:

1. “Add a list” butonuna tıklayın
2. Liste adı girin (örn: “To Do”, “In Progress”, “Done”)
3. Enter tuşuna basın veya “Add” butonuna tıklayın

Kart Ekleme:

1. Bir listenin altındaki “Add a card” butonuna tıklayın
2. Kart başlığı girin
3. Enter tuşuna basın veya “Add” butonuna tıklayın
4. Karta tıklayarak detayları düzenleyebilirsiniz

Kart Detayları:

- Açıklama ekleyin
- Etiket (label) ekleyin
- Dosya (attachment) yükleyin
- Yorum (comment) yapın
- Tarih (due date) belirleyin



Sık Karşılaşılan Sorunlar ve Çözümleri

1. PostgreSQL Bağlantı Hataları

Hata Mesajı:

```
Error: P1001: Can't reach database server at localhost:5432
```

Çözümler:

Çözüm 1: PostgreSQL Servisini Kontrol Edin

Windows:

```
# Servisi başlatın
net start postgresql-x64-16

# Veya Services uygulamasından kontrol edin
```

macOS (Homebrew):

```
# Servis durumunu kontrol edin
brew services list

# Başlatın
brew services start postgresql@15
```

macOS (Postgres.app):

- Uygulamayı açın ve başlatın

Çözüm 2: DATABASE_URL'i Kontrol Edin

.env dosyasındaki DATABASE_URL 'in doğru olduğundan emin olun:

```
# Bağlantıyı test edin
psql "postgres://postgres:postgres123@localhost:5432/ulgen_db"
```

Bağlanamazsanız:

- Kullanıcı adı doğru mu?
- Şifre doğru mu?
- Veritabanı oluşturuldu mu?
- Port 5432 kullanılıyor mu?

Çözüm 3: Veritabanının Var Olduğunu Kontrol Edin

```
# PostgreSQL'e bağlanın
psql -U postgres

# Veritabanlarını listeleyin
\l

# ulgen_db yoksa oluşturun
CREATE DATABASE ulgen_db;

# Çıkış
\q
```

2. Port Çakışmaları

Hata Mesajı:

```
Error: Port 3000 is already in use
```

Çözümler:

Çözüm 1: Portu Kullanan Uygulamayı Kapatın

Windows:

```
# Portu kullanan process'i bulun
netstat -ano | findstr :3000

# Process ID'yi not edin ve sonlandırın
taskkill /PID <process_id> /F
```

macOS/Linux:

```
# Portu kullanan process'i bulun
lsof -i :3000

# Process'i sonlandırın
kill -9 <PID>
```

Çözüm 2: Farklı Bir Port Kullanın

```
# Windows (PowerShell)
$env:PORT=3001; pnpm dev

# macOS/Linux
PORT=3001 pnpm dev
```

3. pnpm Komutları Çalışmıyor

Hata Mesajı:

```
'pnpm' is not recognized as an internal or external command
```

Çözümler:

Çözüm 1: pnpm'i Global Olarak Kurun

```
npm install -g pnpm
```

Çözüm 2: Terminal'i Yeniden Başlatın

Kurulumdan sonra terminal/command prompt'u kapatıp yeniden açın.

Çözüm 3: PATH'i Kontrol Edin

Windows:

```
# pnpm'in yolunu bulun
npm config get prefix

# Bu yol Windows PATH'inde olmalı
# Sistem Özellikleri > Gelişmiş > Ortam Değişkenleri > Path
```

macOS/Linux:

```
# pnpm'in yolunu bulun
which pnpm

# Yoksa PATH'e ekleyin
echo 'export PATH="$HOME/.local/share/pnpm:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

4. Prisma Migration Hataları

Hata Mesajı:

```
Error: P3009: migrate found failed migrations
```

Çözümler:

Çözüm 1: Veritabanını Sıfırlayın (Development)

⚠ Dikkat: Bu komut tüm verileri siler!

```
# Veritabanını sıfırlayın
pnpm db:push --force-reset

# Seed data'yı yeniden yükleyin
pnpm db:seed
```

Çözüm 2: Manuel Veritabanı Temizliği

```
# PostgreSQL'e bağlanın
psql -U postgres

# Veritabanını silin
DROP DATABASE ulgen_db;

# Yeniden oluşturun
CREATE DATABASE ulgen_db;

# Çıkış
\q

# Schema'yı push edin
pnpm db:push

# Seed data yükleyin
pnpm db:seed
```

5. NextAuth Hataları

Hata Mesajı:

```
[next-auth][error][NO_SECRET]
```

Çözüm:

`.env` dosyasında `NEXTAUTH_SECRET` tanımlı olduğundan emin olun:

```
# Yeni bir secret oluşturun
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"

# .env dosyasına ekleyin
NEXTAUTH_SECRET="oluşturulan-secret"
```

Hata Mesajı:

```
[next-auth][error][SIGNIN_EMAIL_ERROR]
```

Çözüm:

1. Email provider doğru yapılandırılmış mı kontrol edin

2. Veritabanı bağlantısı çalışıyor mu kontrol edin
3. User tablosu oluşturulmuş mu kontrol edin:

```
psql -U postgres -d ulgen_db -c "\dt"
```

6. Module Not Found Hataları

Hata Mesajı:

```
Error: Cannot find module 'xyz'
```

Çözüm:

```
# node_modules'u silin
rm -rf node_modules

# pnpm lock dosyasını silin
rm pnpm-lock.yaml

# Yeniden kurun
pnpm install
```

7. Build Hataları

Hata Mesajı:

```
Type error: Property 'xyz' does not exist
```

Çözüm:

```
# Prisma Client'ı yeniden generate edin
pnpm db:generate

# TypeScript cache'i temizleyin
rm -rf .next

# Yeniden build edin
pnpm build
```

8. Yavaş Performans

Çözümler:

1. Development mode'da çalıştığınızdan emin olun:

```
bash
```

```
pnpm dev
```

2. Node.js ve pnpm'i güncelleyin:

```
```bash
Node.js versiyonunu kontrol edin
node -version # v18+ olmalı
```

```
pnpm'i güncelleyin
npm install -g pnpm@latest
```
```

1. Veritabanı indekslerini kontrol edin:

- Prisma schema'da `@@index` tanımları var mı?

2. Tarayıcı cache'ini temizleyin:

- Chrome: Ctrl+Shift+Delete (Windows) / Cmd+Shift+Delete (macOS)

Komut Referansı

Geliştirme Komutları

| Komut | Açıklama | Ne Zaman Kullanılır |
|--------------------------|-------------------------------|--------------------------|
| <code>pnpm dev</code> | Development server'ı başlatır | Geliştirme yaparken |
| <code>pnpm build</code> | Production build oluşturur | Deploy öncesi |
| <code>pnpm start</code> | Production server'ı başlatır | Build sonrası çalıştırma |
| <code>pnpm lint</code> | ESLint ile kod kontrolü | Kod kalitesi kontrolü |
| <code>pnpm format</code> | Prettier ile kod formatlama | Kod düzenleme sonrası |

Veritabanı Komutları

| Komut | Açıklama | Ne Zaman Kullanılır |
|-------------------------------|-----------------------------|---|
| <code>pnpm db:generate</code> | Prisma Client oluşturur | Schema değişikliği sonrası |
| <code>pnpm db:push</code> | Schema'yı DB'ye push eder | İlk kurulum veya schema değişikliği |
| <code>pnpm db:seed</code> | Demo data yükler | İlk kurulum veya test için |
| <code>pnpm db:studio</code> | Prisma Studio'yu açar | Veritabanını görsel olarak incelemek için |
| <code>pnpm db:reset</code> | DB'yi sıfırlar ve seed eder | Temiz başlangıç için |

Prisma Studio

Veritabanınızı görsel olarak yönetmek için:

```
pnpm db:studio
```

Tarayıcıda `http://localhost:5555` adresinde açılır.

Ek Bilgiler

Proje Yapısı

```
trello-clone/
├── src/
│   ├── app/           # Next.js App Router sayfaları
│   ├── components/    # React bileşenleri
│   ├── lib/           # Yardımcı fonksiyonlar
│   └── styles/        # CSS/Tailwind stilleri
├── prisma/
│   ├── schema.prisma  # Veritabanı şeması
│   └── seed.ts        # Seed data scripti
├── public/            # Statik dosyalar
├── .env               # Environment variables (GİT'E EKLEMEYİN!)
├── .env.example       # Örnek env dosyası
├── package.json       # Proje bağımlılıkları
└── tsconfig.json      # TypeScript konfigürasyonu
```

Teknoloji Stack'i

- **Framework:** Next.js 14 (App Router)
- **UI:** React 18, Tailwind CSS
- **Veritabanı:** PostgreSQL
- **ORM:** Prisma
- **Authentication:** NextAuth.js
- **Drag & Drop:** @dnd-kit
- **Form Validation:** Zod
- **TypeScript:** Type-safe development

Aşama 2 ve 3'te Neler Gelecek?

Aşama 2: Gelişmiş Özellikler

- ☒ Gerçek zamanlı işbirliği (WebSocket)
- ☒ Dosya yükleme ve önizleme
- ☒ Gelişmiş arama ve filtreleme
- ☒ Board şablonları
- ☒ Aktivite timeline'ı
- ☒ Bildirimler sistemi

Aşama 3: Kurumsal Özellikler

- ☒ Takım yönetimi ve roller
- ☒ Workspace'ler
- ☒ Gelişmiş raporlama ve analytics
- ☒ API ve webhook'lar
- ☒ Mobil uygulama
- ☒ Entegrasyonlar (Slack, GitHub, vb.)

Destek ve Yardım

Dokümantasyon:

- [Next.js Docs](https://nextjs.org/docs) (https://nextjs.org/docs)
- [Prisma Docs](https://www.prisma.io/docs) (https://www.prisma.io/docs)
- [NextAuth.js Docs](https://next-auth.js.org) (https://next-auth.js.org)
- [Tailwind CSS Docs](https://tailwindcss.com/docs) (https://tailwindcss.com/docs)

Sorun Bildirimi:

- GitHub Issues: `<repository-url>/issues`
- Email: support@ulgen.com

Topluluk:

- Discord: `<discord-invite-link>`
- Twitter: @ulgen_dev



Tebrikler!

ULGEN Trello Clone projesini başarıyla kurdunuz! Artık:

- ✓ Veritabanı çalışıyor
- ✓ Uygulama ayakta
- ✓ Giriş yapabiliyorsunuz
- ✓ Board'lar oluşturabiliyorsunuz

Keyifli kodlamalar! 🚀

Son Güncelleme: Aralık 2024

Versiyon: 1.0.0

Lisans: MIT